

```
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.manifold import TSNE

from keras.datasets import mnist

(train_X, train_y), (test_X, test_y) = mnist.load_data()

x_train = train_X[:1000]
y_train = train_y[:1000]
print(x_train.shape)
(1000, 28, 28)

x_mnist = np.reshape(x_train, [x_train.shape[0], x_train.
shape[1]*x_train.shape[2]])
print(x_mnist.shape)
(1000, 784)

tsne = TSNE(n_components=2, verbose=1, random_state=123)
k = tsne.fit_transform(x_mnist)
dfmnist = pd.DataFrame()
dfmnist["y"] = y_train
dfmnist["dimension-1"] = k[:,0]
dfmnist["dimension-2"] = k[:,1]
sns.scatterplot(x="dimension-1", y="dimension-2",
hue=dfmnist.y,
palette=sns.color_palette("Set2", 10),
data=dfmnist).set(title="MNIST data T-SNE
projection")
plt.show()
```

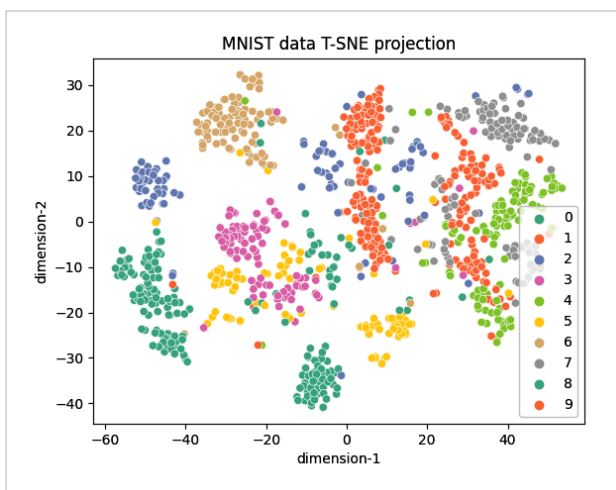


Figure 5: t-SNE for MNIST data set with 1000 samples

As can be seen, the distribution does not cluster better with 1000 samples (Figure 5), but increasing the number of samples to 3000 gives the plot shown in Figure 6. It can be seen that the clustering of data points is better visualised in this plot.

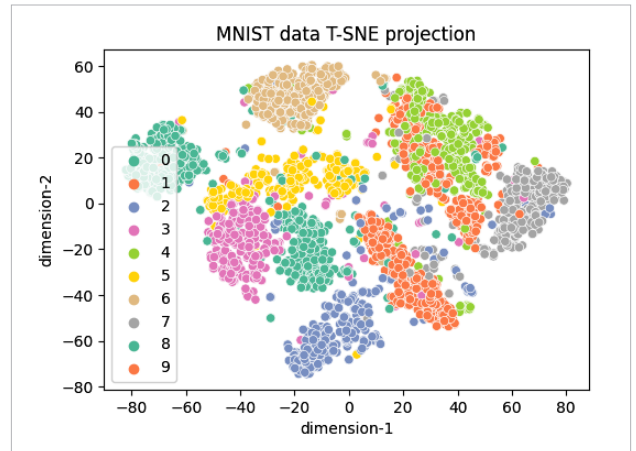


Figure 6: t-SNE for MNIST data set with 3000 samples

Kernel PCA

This method can be applied to data sets that are non-linearly separable, as the PCA method works only on linearly separable data. Kernel PCA separates the non-linear samples in the data set by projecting them into the higher dimensional space. It uses a kernel function to transform dimensions of the original data set into a higher dimensional space where the samples are linearly separable. Post that, the PCA method can be applied on the transformed data to generate the Eigen vectors and Eigen values to perform dimensionality reduction.

While the kernel function transforms the original dimensions into the higher dimensional space, the samples in the original dimensions increase the number of variables in higher dimensions and hence increase the computational complexity of the problem. This problem can be solved by using the kernel trick, which helps to generate the kernel matrix in lesser variables compared to the number of variables if these are directly transformed to a higher dimensional space. A commonly used kernel trick is the Gaussian kernel or polynomial kernel. The kernel trick is similar to the one used in support vector machines.

The code sample given below illustrates how PCA fails to separate the non-linear data and Kernel PCA succeeds in separating it. The *sklearn* methods used earlier are applied on the standard moons data set to illustrate this:

```
from sklearn.decomposition import KernelPCA
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
```

```
train_X, train_y = make_moons(n_samples = 500, noise = 0.03,
random_state = 510)
```

```
plt.scatter(train_X[:, 0], train_X[:, 1], c = train_y*10)
plt.show()
```